Akamata Tutorial — Build a Todo List API from scratch

This tutorial is for readers **new to both Zig and Akamata**. You'll go from an empty directory to a **production-ready Todo list API + web UI** in one sitting.

Reading time: **60–90 minutes**. Skip sections you already know — each one opens with a "goal" so you can decide.

What you'll build

A small web app managing `Todo` resources:

- `GET /api/todos` List todos (incomplete first)
- `POST /api/todos` Create a todo (title required, priority optional)
- `GET /api/todos/:id` Fetch one
- `PUT /api/todos/:id` Update (rename, mark done)
- `DELETE /api/todos/:id` Remove
- `GET /health` Liveness check
- `GET /metrics` Prometheus metrics
- `GET /` HTML UI (works in the browser)

Backends:

- **Local development**: SQLite file (`file:todo.db`)
- **Production**: Cloudflare D1 on Workers

The same single source tree builds and deploys to both.

Table of contents

---

0. Prerequisites and setup

Goal

- Confirm every tool is installed
- Verify `akamata help` runs

Required

- **Zig 0.16.0** — check via `zig version`. If missing, install from ziglang.org/download (extract the official tarball into `$HOME/.local` and add it to `PATH`).
- macOS or Linux terminal (Windows: use WSL2)
- `curl`

Recommended (required for Workers deploy)

- **Node.js 22+** with `npx wrangler` (Cloudflare Workers SDK)
- A **Cloudflare account** (free tier is fine)
- A modern browser

Verify

```
zig version
# Expected: 0.16.0 or 0.16.0-dev.NNN+xxxxxxx
```

Get Akamata itself

Akamata's framework code and the `akamata` CLI live in the same repo.

```
git clone https://github.com/yourorg/Akamata
cd Akamata
zig build cli
```

After it builds you'll have `zig-out/bin/akamata`. Add it to your `PATH` so later commands are short:

```
# temporary
export PATH="$PWD/zig-out/bin:$PATH"

# persistent (zsh)
echo 'export PATH="'"$PWD"'/zig-out/bin:$PATH"' >> ~/.zshrc
```

Smoke-test:

```
akamata help
```

```
Usage: akamata <command> [args]

Commands:
  init <name> [--target=native|workers|containers|both]
      Scaffold a new Akamata app.
  build [--workers|--containers]
      Build the current app (native by default).
  ...
```

> **Gotcha**: if you see `command not found: akamata` after adding to `.zshrc`, open a fresh terminal — your existing shell hasn't reloaded the file. Alternatively call the full path: `./zig-out/bin/akamata help`.

---

1. Scaffold a project

Goal

- Use `akamata init` to generate a `mytodo` skeleton
- Understand the resulting directory layout

Command

Move **outside** the Akamata repo to wherever you keep your own projects:

```
cd ~/projects        # any working directory
akamata init mytodo --target=both
```

`--target=both` means "generate files for native binaries AND Cloudflare Workers".

Expected output

```
Created mytodo/

Next steps:
  cd mytodo
  zig build run           # native dev server
```

Layout

```
cd mytodo
tree -a -L 2
```

```
mytodo/
├── .gitignore
├── README.md
├── build.zig                  # build configuration
├── build.zig.zon              # package manifest (depends on Akamata)
├── deploy/
│   ├── wrangler.toml          # Cloudflare Workers config
│   └── worker/
│       └── index.mjs          # JS host that loads the wasm
└── src/
    ├── main.zig               # the app (you'll edit this)
    └── worker.zig             # Workers entry point
```

What each file does

File	Role
`build.zig`	Zig build configuration. Flags like `-Dbackend=workers` are parsed here
`build.zig.zon`	Package manifest including the Akamata dependency
`src/main.zig`	Native entry point. You'll spend 90% of your time here
`src/worker.zig`	Workers entry point — just calls into `main.zig`
`deploy/wrangler.toml`	D1 bindings + env vars
`deploy/worker/index.mjs`	JS shim that loads the wasm and bridges D1

> **Rule of thumb**: handlers go in `src/main.zig`. You won't touch the other files for a long time.

---

2. Run the scaffold and read the structure

Goal

- Build and run the unmodified scaffold
- Hit it with curl
- Read the structure of `src/main.zig`

Build and start

```
zig build run
```

First build takes 1–2 minutes (Akamata + SQLite C amalgamation). Subsequent builds are incremental and take seconds.

Expected output:

```
info: mytodo listening on :8080
info: akamata listening on http://0.0.0.0:8080/
```

Smoke-test

In another terminal:

```
curl -sS http://127.0.0.1:8080/
```

```
{"name":"mytodo","endpoints":{"health":"GET /health","list":"GET /notes","create":"POST /not
```

The scaffold ships with a `Note` model; we'll replace it with `Todo`.

`Ctrl-C` to stop the server.

Read src/main.zig

Open `src/main.zig`. It's split into sections:

```
// ===== Model ==================================================
pub const Note = struct { ... };   // the data model
const Notes = am.model.repo(Note); // CRUD helpers for Note

// ===== App state ==============================================
pub const State = struct { db: am.db.Db };  // shared state across handlers
pub const all_models = [_]am.model.TableDef{ ... };

// ===== Handlers ===============================================
fn index(c: *Ctx) !void { ... }    // HTTP handlers
fn createNote(c: *Ctx) !void { ... }

// ===== Wiring =================================================
pub fn registerRoutes(app: *am.App(State)) !void { ... }
pub fn buildState(alloc: std.mem.Allocator) !State { ... }
pub fn main(...) !void { ... }     // entry point
```

Zig basics (for total beginners)

```
const std = @import("std");
const am  = @import("akamata");
```

`const x = value;` declares an **immutable variable**. `@import` loads a module. `am` is a short alias for Akamata.

```
pub const Todo = struct {
    id: ?i64 = null,
    title: []const u8,
};
```

- `pub` exports the struct from this module

- `?i64` is an **optional** i64 — holds an i64 or null
- `[]const u8` is a **byte slice** — the canonical string type
- `= null` / `= ""` provide default values

```
fn hello(c: *Ctx) !void {
    try c.text("Hello");
}
```

- `*Ctx` is a pointer to the request context (one per request)
- `!void` is "either void or an error"
- `try` propagates errors to the caller — the equivalent of Rust's `?`

That's 90% of what you need to read this tutorial.

---

3. Define the Todo model

Goal

- Replace `Note` with `Todo`
- Add `priority` and `completed` fields
- Set up an index and a default

Schema

Column	Zig type	SQL type	Note
`id`	`?i64 = null`	INTEGER PRIMARY KEY	auto-increment
`title`	`[]const u8`	TEXT NOT NULL	required
`priority`	`i32 = 3`	INTEGER NOT NULL	1=high, 2=mid, 3=low
`completed`	`bool = false`	INTEGER NOT NULL	done flag
`created_at`	`?i64 = null`	INTEGER (unix epoch)	created timestamp

Edit

In `src/main.zig`, replace the entire `Note` section with the Todo model below. You can delete the old `Note`-related code entirely:

```
// ===== Model ==================================================

pub const Todo = struct {
    id: ?i64 = null,
    title: []const u8,
    priority: i32 = 3,
    completed: bool = false,
    created_at: ?i64 = null,

    pub const __schema = .{
        .table = "todos",
        .primary_key = "id",
        .indexes = .{
            // Speeds up the canonical "open todos, highest priority first" query.
            .{ .{ "completed", "priority", "created_at" }, .index },
        },
        .defaults = .{
            .created_at = "unixepoch()",
        },
        .validates = .{
            .title    = .{ am.model.rule.required, am.model.rule.max_len(200) },
            .priority = .{ am.model.rule.range(1, 3) },
        },
    };
};

const Todos = am.model.repo(Todo);
```

What each `__schema` field means

Key	Effect
`table`	SQL table name (defaults to lowercased struct name + "s")
`primary_key`	PK field name (defaults to `"id"`)
`indexes`	Index definitions — end each tuple with `.unique` or `.index`
`defaults`	SQL `DEFAULT (expr)` clauses. `unixepoch()` is built into SQLite
`validates`	Validation rules (see § 6)

Update `all_models`

A few lines below, update the list:

```
pub const all_models = [_]am.model.TableDef{
    am.model.tableDef(Todo),
};
```

> **Remember**: every new model must be appended to `all_models`. The auto-migrator reads this list to issue `CREATE TABLE` statements.

---

4. Auto-migrate the DB

Goal

- Understand that the scaffold auto-creates the table at boot
- See how schema drift triggers `ALTER TABLE` automatically

The generated `buildState` already calls the migrator. Switching from `Note` to `Todo` just means rebuilding and restarting.

Reset the old DB and start fresh

```
rm -f mytodo.db                # delete the Note-era DB
zig build run
```

If no errors fire, you're good. Optionally inspect the new schema:

```
sqlite3 mytodo.db ".schema"
```

```
CREATE TABLE schema_migrations (...);
CREATE TABLE todos (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  priority INTEGER NOT NULL,
  completed INTEGER NOT NULL,
  created_at INTEGER DEFAULT (unixepoch())
) STRICT;
CREATE INDEX todos_completed_priority_created_at_idx ON todos (completed, priority, created_
```

Drift example (optional)

If you later add `notes: []const u8 = ""` to the struct, restart and the migrator logs:

```
warn: migrate.diff found drift, applying: ALTER TABLE todos ADD COLUMN notes TEXT NOT NULL D
```

DROP COLUMN is dangerous, so it's never automatic — you'll see a warning but the column stays.

> **Under the hood**: `am.model.migrate.diff()` compares `PRAGMA table_info()` output against `__schema` and generates the minimum `ALTER TABLE ADD COLUMN` / `CREATE INDEX` to make them match.

---

5. Write the CRUD handlers

Goal

- Implement five handlers (`list`, `create`, `show`, `update`, `delete`)

- Learn `Todos.find/all/where/create/save/delete`

Replace the Handlers section entirely:

```
// ===== Handlers ==================================================

const Ctx = am.Context(State);

fn index(c: *Ctx) !void {
    // We'll return the HTML UI in chapter 7. For now, return a JSON map.
    try c.json(.{
        .name = "mytodo",
        .endpoints = .{
            .list   = "GET /api/todos",
            .create = "POST /api/todos  { title, priority? }",
            .show   = "GET /api/todos/:id",
            .update = "PUT /api/todos/:id  { title?, priority?, completed? }",
            .delete = "DELETE /api/todos/:id",
        },
    }, 200);
}

fn health(c: *Ctx) !void {
    var stmt = c.db().prepare("SELECT 1") catch return c.serverError("db unavailable");
    defer stmt.deinit();
    _ = stmt.step() catch return c.serverError("db unavailable");
    try c.json(.{ .status = "ok" }, 200);
}

fn listTodos(c: *Ctx) !void {
    // open first, then priority, then newest — the order the index covers.
    const rows = try Todos.queryRaw(c.db(), c.arena,
        "SELECT id, title, priority, completed, created_at FROM todos " ++
        "ORDER BY completed ASC, priority ASC, created_at DESC LIMIT 200",
        .{},
    );
    try c.json(.{ .todos = rows }, 200);
}

fn createTodo(c: *Ctx) !void {
    // c.input parses JSON + runs Todo's __schema.validates rules.
    // On failure it writes 400 / 422 itself and returns null.
    const todo = (try c.input(Todo)) orelse return;
    const created = try Todos.create(c.db(), c.arena, todo);
    try c.json(created, 201);
}

fn showTodo(c: *Ctx) !void {
    const id = c.req.paramAs(i64, "id") catch return c.badRequest("invalid id");
    const t = (try Todos.find(c.db(), c.arena, id)) orelse return c.notFound();
    try c.json(t, 200);
}

fn updateTodo(c: *Ctx) !void {
    const id = c.req.paramAs(i64, "id") catch return c.badRequest("invalid id");
    var t = (try Todos.find(c.db(), c.arena, id)) orelse return c.notFound();

    // Accept a partial update via an all-optional struct.
    const Patch = struct {
```

```
        title: ?[]const u8 = null,
        priority: ?i32 = null,
        completed: ?bool = null,
    };
    const patch = c.req.json(Patch) catch return c.badRequest("invalid JSON");
    if (patch.title)     |v| t.title = try c.arena.dupe(u8, v);
    if (patch.priority)  |v| t.priority = v;
    if (patch.completed) |v| t.completed = v;

    try Todos.save(c.db(), c.arena, &t);
    try c.json(t, 200);
}

fn deleteTodo(c: *Ctx) !void {
    const id = c.req.paramAs(i64, "id") catch return c.badRequest("invalid id");
    try Todos.delete(c.db(), id);
    try c.json(.{ .deleted = id }, 200);
}
```

Things worth noticing

`c.db()` and `c.arena`

- `c.db()` is a shortcut for `c.state().db` — the **handle to issue SQL**
- `c.arena` is the **request-scoped allocator**. Anything you `alloc` from it lives until the response is sent, then it all goes away. **You never need to free it manually**.

`c.input(Todo)`

```
const todo = (try c.input(Todo)) orelse return;
```

In a single call it:

1. Parses the JSON body
2. Runs `Todo.__schema.validates`
3. On failure, writes 400 (parse error) or 422 (validation error) and returns `null`
4. On success, returns the parsed value

`orelse return` means "bail out cleanly when the response is already written".

Why `queryRaw` instead of `Todos.all()`?

`Todos.all()` orders by primary key descending. We needed business-specific ordering (open first, priority, then newest), so we dropped to the raw-SQL escape hatch. The result is still mapped back into `Todo` structs.

Wire the routes

Replace `registerRoutes`:

```
pub fn registerRoutes(app: *am.App(State)) !void {
    _ = try app.useAll(am.mw.recover(State));
    _ = try app.useAll(am.mw.logger(State));

    _ = try app.get("/",                index);
    _ = try app.get("/health",          health);
    _ = try app.get("/api/todos",       listTodos);
    _ = try app.post("/api/todos",      createTodo);
    _ = try app.get("/api/todos/:id",   showTodo);
    _ = try app.put("/api/todos/:id",   updateTodo);
    _ = try app.delete("/api/todos/:id", deleteTodo);
}
```

Run it

```
zig build run
```

In another terminal:

```
# 1. Create
curl -sS -X POST -H 'content-type: application/json' \
  -d '{"title":"買い物","priority":1}' \
  http://127.0.0.1:8080/api/todos
```

```
{"id":1,"title":"買い物","priority":1,"completed":false,"created_at":1779999999}
```

```
# 2. List
curl -sS http://127.0.0.1:8080/api/todos
```

```
{"todos":[{"id":1,"title":"買い物","priority":1,"completed":false,"created_at":1779999999}]}
```

```
# 3. Complete
curl -sS -X PUT -H 'content-type: application/json' \
  -d '{"completed":true}' \
  http://127.0.0.1:8080/api/todos/1
```

```
{"id":1,"title":"買い物","priority":1,"completed":true,"created_at":1779999999}
```

```
# 4. Delete
curl -sS -X DELETE http://127.0.0.1:8080/api/todos/1
```

```
{"deleted":1}
```

---

6. Add validation

Goal

- Verify the validator rejects bad input
- Learn to write a custom validator

Built-in rules

You already declared:

- `title` is required + max 200 chars
- `priority` is 1..=3

Try violating them:

```
# title empty → 422
curl -sS -w "\nHTTP %{http_code}\n" -X POST \
  -H 'content-type: application/json' \
  -d '{"title":"","priority":1}' \
  http://127.0.0.1:8080/api/todos
```

```
{"error_kind":"validation","errors":[{"field":"title","rule":"required","message":"is requir
HTTP 422
```

```
# priority out of range → 422
curl -sS -w "\nHTTP %{http_code}\n" -X POST \
  -H 'content-type: application/json' \
  -d '{"title":"x","priority":99}' \
  http://127.0.0.1:8080/api/todos
```

```
{"error_kind":"validation","errors":[{"field":"priority","rule":"range","message":"must be b
HTTP 422
```

```
# garbage JSON → 400
curl -sS -w "\nHTTP %{http_code}\n" -X POST \
  -H 'content-type: application/json' \
  -d 'not json' \
  http://127.0.0.1:8080/api/todos
```

```
{"error_kind":"bad_request","message":"invalid JSON body"}
HTTP 400
```

Custom validator

Say you want "title must not contain `<script>`":

```
fn noScriptTag(value: []const u8, _: std.mem.Allocator) ?[]const u8 {
    if (std.mem.indexOf(u8, value, "<script>") != null) {
        return "must not contain <script>";
    }
    return null;
}

pub const Todo = struct {
    // ...
    pub const __schema = .{
        // ...
        .validates = .{
            .title    = .{
                am.model.rule.required,
                am.model.rule.max_len(200),
                am.model.rule.custom(noScriptTag),
            },
            .priority = .{ am.model.rule.range(1, 3) },
        },
    };
};
```

- Return `null` for success
- Return a string for the error message

For integer fields use `am.model.rule.customInt(fn)`.

---

7. Bundle an HTML UI with embedFile

Goal

- Browse to the app and use it without curl
- Bake HTML/CSS/JS into the wasm/native binary
- Serve HTML vs JSON via the `Accept` header

Create `src/index.html`

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>mytodo</title>
<style>
  body { font-family: system-ui, sans-serif; max-width: 640px; margin: 2rem auto; padding: 0
  h1 { margin: 0 0 1rem; }
  form { display: flex; gap: 0.5rem; margin-bottom: 1.5rem; }
  input[type="text"] { flex: 1; padding: 0.4rem 0.6rem; font: inherit; border: 1px solid #99
  select, button { padding: 0.4rem 0.6rem; font: inherit; border: 1px solid #999; border-rad
  button[type="submit"] { background: #2563eb; color: #fff; border-color: #2563eb; cursor: p
  ul { list-style: none; padding: 0; }
  li { display: flex; align-items: center; gap: 0.5rem; padding: 0.5rem; border-bottom: 1px
  li.done .title { text-decoration: line-through; color: #888; }
  .pri { width: 1.4rem; height: 1.4rem; border-radius: 50%; display: inline-block; }
  .pri-1 { background: #ef4444; }
  .pri-2 { background: #f59e0b; }
  .pri-3 { background: #9ca3af; }
  .title { flex: 1; }
  .del { color: #b91c1c; background: transparent; border: none; cursor: pointer; }
  .err { color: #b91c1c; font-size: 0.9rem; min-height: 1.2rem; }
</style>
</head>
<body>
<h1>mytodo</h1>
<form id="f">
  <input type="text" id="title" placeholder="What needs doing?" required maxlength="200">
  <select id="priority">
    <option value="1">High</option>
    <option value="2">Med</option>
    <option value="3" selected>Low</option>
  </select>
  <button type="submit">Add</button>
</form>
<div id="err" class="err"></div>
<ul id="list"></ul>

<script>
const $ = (id) => document.getElementById(id);
const api = async (method, path, body) => {
  const r = await fetch(path, {
    method,
    headers: body ? { "content-type": "application/json" } : {},
    body: body ? JSON.stringify(body) : null,
  });
  if (!r.ok) {
    const j = await r.json().catch(() => ({ message: r.statusText }));
    throw new Error(j.message || JSON.stringify(j.errors));
  }
  return r.json();
};
```

```
async function refresh() {
  try {
    $("err").textContent = "";
    const { todos } = await api("GET", "/api/todos");
    $("list").innerHTML = todos.map(t => `
      <li class="${t.completed ? "done" : ""}" data-id="${t.id}">
        <input type="checkbox" ${t.completed ? "checked" : ""} class="chk">
        <span class="pri pri-${t.priority}" title="priority ${t.priority}"></span>
        <span class="title">${escapeHtml(t.title)}</span>
        <button class="del">delete</button>
      </li>
    `).join("");
    [...$("list").children].forEach((li, i) => {
      const id = parseInt(li.dataset.id, 10);
      li.querySelector(".chk").onchange = (e) =>
        api("PUT", `/api/todos/${id}`, { completed: e.target.checked }).then(refresh);
      li.querySelector(".del").onclick = () =>
        api("DELETE", `/api/todos/${id}`).then(refresh);
    });
  } catch (e) {
    $("err").textContent = e.message;
  }
}

function escapeHtml(s) {
  return s.replace(/[&<>"']/g, c => ({ "&":"&amp;","<":"&lt;",">":"&gt;",'"':"&quot;","'":"&
}

$("f").onsubmit = async (e) => {
  e.preventDefault();
  try {
    $("err").textContent = "";
    await api("POST", "/api/todos", {
      title: $("title").value,
      priority: parseInt($("priority").value, 10),
    });
    $("title").value = "";
    refresh();
  } catch (err) {
    $("err").textContent = err.message;
  }
};

refresh();
</script>
</body>
</html>
```

Embed it in `src/main.zig`

Just above the Handlers section, add:

```
const index_html = @embedFile("index.html");
```

`@embedFile` reads the file at compile time and gives you a `[]const u8` of its contents — **the file is never read at runtime**, so nothing extra needs to be deployed.

Update the `index` handler

```
fn index(c: *Ctx) !void {
    // Browsers send `Accept: text/html` — serve the UI.
    // curl / fetch send `*/*` or `application/json` — serve the API map.
    const accept = c.req.header("accept") orelse "";
    if (std.mem.indexOf(u8, accept, "text/html") != null) {
        return c.html(index_html);
    }
    try c.json(.{
        .name = "mytodo",
        .endpoints = .{
            .list   = "GET /api/todos",
            .create = "POST /api/todos  { title, priority? }",
            .show   = "GET /api/todos/:id",
            .update = "PUT /api/todos/:id  { title?, priority?, completed? }",
            .delete = "DELETE /api/todos/:id",
        },
    }, 200);
}
```

Try it

```
zig build run
```

Open http://127.0.0.1:8080/ in a browser. You can add, check off, and delete todos through the UI.

```
mytodo
[          What needs doing?          ] [Low v] [Add]

○ ● Groceries                          [delete]
✓ ● Write report                       [delete]
```

(The coloured dot is priority, the checkbox toggles completed.)

Verify the negotiation

```
# curl defaults to Accept: */* → JSON
curl -sS http://127.0.0.1:8080/

# emulate a browser → HTML
curl -sS -H 'Accept: text/html' http://127.0.0.1:8080/ | head -3
```

---

8. Routing and middleware

Goal

- Understand the role of `recover` and `logger`
- Add `accessLog` and `metrics` for production-ready observability
- (Optional) group routes with `basePath`

Order matters

Middleware applied via `useAll` runs in declaration order around every request. Order matters:

```
_ = try app.useAll(am.mw.recover(State));   // 1. trap panics → 500
_ = try app.useAll(am.mw.logger(State));    // 2. log
```

`recover` is outermost so we can return a 500 to the user even when a deeper layer panics.

Add requestId, accessLog, and metrics

Expand `registerRoutes`:

```
var metrics_counters: am.MetricsCounters = .{};

pub fn registerRoutes(app: *am.App(State)) !void {
    _ = try app.useAll(am.mw.recover(State));
    _ = try app.useAll(am.mw.requestId(State));           // X-Request-ID
    _ = try app.useAll(am.mw.accessLog(State, .json));    // structured one-line log
    _ = try app.useAll(am.mw.metrics(State, &metrics_counters));

    _ = try app.get("/",        index);
    _ = try app.get("/health",  health);
    _ = try app.get("/metrics", am.mw.metricsHandler(State, &metrics_counters));

    _ = try app.get("/api/todos",        listTodos);
    _ = try app.post("/api/todos",       createTodo);
    _ = try app.get("/api/todos/:id",    showTodo);
    _ = try app.put("/api/todos/:id",    updateTodo);
    _ = try app.delete("/api/todos/:id", deleteTodo);
}
```

> **Important**: `metrics_counters` must be a **module-level** `var` (one that outlives any single request). Putting it inside `registerRoutes` would create a dangling pointer. Put it at file scope, near `State`.

Confirm

`zig build run`, send a few requests, then:

```
curl -sS http://127.0.0.1:8080/metrics | head -20
```

```
# HELP akamata_requests_total Total HTTP requests served.
# TYPE akamata_requests_total counter
akamata_requests_total 12
# HELP akamata_requests_in_flight Requests currently being processed.
# TYPE akamata_requests_in_flight gauge
akamata_requests_in_flight 1
# HELP akamata_requests_by_status Requests broken down by HTTP status class.
# TYPE akamata_requests_by_status counter
akamata_requests_by_status{class="1xx"} 0
akamata_requests_by_status{class="2xx"} 10
akamata_requests_by_status{class="3xx"} 0
akamata_requests_by_status{class="4xx"} 2
akamata_requests_by_status{class="5xx"} 0
...
```

Your `zig build run` terminal prints one JSON line per request:

```
{"ts_unix_us":1779999999000000,"req_id":"a64d6f73-2b0e-4ad1-9aa3-8c0f4f2c5d6e","ip":"-","met
```

`req_id` is also echoed on `X-Request-ID` so you can correlate front-end errors with server logs.

More detail in docs/observability.md.

---

9. Deploy to Cloudflare Workers + D1

Goal

- Build the same source as a Workers wasm
- Create a D1 database and auto-migrate it via Akamata
- Verify production via the live URL

9.1 Try Workers mode locally

`wrangler dev --local` runs the wasm inside Miniflare with a simulated D1.

```
zig build -Dbackend=workers -Doptimize=ReleaseSmall

cd deploy
wrangler dev --local --port 18080
```

Excerpt:

```
⛅ wrangler 4.93.1
Your Worker has access to the following bindings:
Binding                          Resource
env.DB (mytodo)                  D1 Database             local
[wrangler:info] Ready on http://localhost:18080
```

Hit it:

```
curl -sS http://127.0.0.1:18080/health
```

```
{"status":"ok"}
```

POST works too. Note: **this writes to a simulated D1, not your SQLite file**.

> **What's happening**: `src/worker.zig` exports a wasm function called `handle_fetch`. The JS shim in `deploy/worker/index.mjs` calls it via JSPI (JavaScript Promise Integration) so that D1's `env.DB.prepare(sql).run()` looks synchronous from Zig. See docs/db-backends.md for the full mechanism.

`Ctrl-C` to stop.

9.2 Deploy to production Cloudflare

```
# back to project root
cd ..

# one-shot deploy
akamata deploy --workers \
  --config=deploy/wrangler.toml \
  --migrate=<(./zig-out/bin/mytodo --print-schema)
```

What this single command does:

1. Reads the `[[d1_databases]]` block from `wrangler.toml`
2. Sees the placeholder `database_id`, runs `wrangler d1 create`, writes the real UUID back to the file
3. Runs `mytodo --print-schema` to dump current DDL
4. Pipes it into `wrangler d1 execute --remote`
5. Runs `zig build -Dbackend=workers -Doptimize=ReleaseSmall`
6. Runs `wrangler deploy`

Excerpt:

```
==> akamata: provisioning D1 "mytodo" (database_id is placeholder)
==> akamata: resolved D1 "mytodo" (id=abcdef12-3456-7890-...)
==> akamata: wrote new database_id back to deploy/wrangler.toml
==> akamata: applying ... to remote D1 "mytodo"
==> akamata: building wasm (ReleaseSmall)
==> akamata: wrangler deploy
...
Uploaded mytodo (2.33 sec)
Deployed mytodo triggers (0.96 sec)
  https://mytodo.<your-subdomain>.workers.dev
```

Open that final URL in a browser — you should see the same UI as locally.

9.3 Verify

```
URL=https://mytodo.<your-subdomain>.workers.dev

curl -sS $URL/health
curl -sS -X POST -H 'content-type: application/json' \
  -d '{"title":"first prod todo","priority":1}' \
  $URL/api/todos
curl -sS $URL/api/todos
```

All 200 — you're live.

> **Troubleshooting**:
>
> - `Did you mean ...?` → typo in the CLI subcommand
> - `D1_EXEC_ERROR` → schema drift. Run `wrangler d1 execute mytodo --remote --command="DROP TABLE todos"` then re-deploy with `--migrate=...`

---

10. Production observability (metrics + logs)

Goal

- Read `/metrics` directly when no Prometheus is available
- Wire it into Prometheus if you have one

10.1 Reading `/metrics`

```
akamata_requests_total 5891
akamata_requests_in_flight 0
akamata_requests_by_status{class="2xx"} 5700
akamata_requests_by_status{class="4xx"} 188
akamata_requests_by_status{class="5xx"} 3
akamata_requests_by_method{method="GET"} 4502
akamata_requests_by_method{method="POST"} 1280
akamata_request_latency_seconds_bucket{le="0.0001"} 4810
akamata_request_latency_seconds_bucket{le="0.001"} 5722
akamata_request_latency_seconds_bucket{le="0.01"} 5891
akamata_request_latency_seconds_bucket{le="+Inf"} 5891
akamata_request_latency_seconds_count 5891
akamata_request_latency_seconds_sum 0.872413
akamata_process_resident_memory_bytes 2621440
akamata_process_uptime_seconds 1234
```

Quick math you can do by eye:

- **Average latency** = `sum / count` = 0.872 / 5891 ≈ **148 µs**
- **Error rate** = `(4xx + 5xx) / total` = 191 / 5891 ≈ **3.2 %**
- **Uptime** is the heartbeat you actually trust

10.2 Prometheus + Grafana

```
scrape_configs:
  - job_name: mytodo
    metrics_path: /metrics
    scrape_interval: 15s
    static_configs:
      - targets: ['mytodo.example.com:8080']
```

PromQL examples:

```
# req/s over the last minute
rate(akamata_requests_total[1m])

# P99 latency
histogram_quantile(0.99, sum by (le) (rate(akamata_request_latency_seconds_bucket[5m])))

# error rate
sum(rate(akamata_requests_by_status{class="5xx"}[5m]))
  / sum(rate(akamata_requests_total[5m]))
```

More in docs/observability.md.

10.3 Persist access logs

`am.mw.accessLog(State, .json)` writes to stderr. In production redirect:

```
./zig-out/bin/mytodo 2> >(tee -a /var/log/mytodo.log >&2)
```

On Workers, use `wrangler tail` for live tailing or Logpush to push to Cloudflare Logpush sinks.

---

11. Common problems and debugging

`error: cannot find file` at build time

- Did you create `src/index.html`?
- Is the `@embedFile` path correct? It's **relative to `src/main.zig`**.

`SuspendError` under `wrangler dev`

```
SuspendError: trying to suspend without WebAssembly.promising
```

The JS host (`deploy/worker/index.mjs`) doesn't have the `WebAssembly.promising` setup. Use the `akamata init`-generated template — it has it. If you've hand-edited an older template, regenerate it.

Server unresponsive

You probably wrote a `while (true)` somewhere. Akamata is one-thread-per-request so one stuck handler just consumes one worker, but if it accumulates the throughput drops.

`BodyTooLarge` in logs

A client sent an over-sized POST. Default ceiling is 8 MB. Raise it with:

```
try app.serve(.{ .port = 8080, .max_body_bytes = 32 * 1024 * 1024 });
```

Migration fails

- Local: `rm mytodo.db` and start over
- Workers + D1: `wrangler d1 execute mytodo --remote --command="DROP TABLE todos"` then re-deploy

Hot reload

Zig is compiled, so you need `Ctrl-C` + `zig build run`. For auto-restart on file changes, use `entr`:

```
ls src/*.zig | entr -r zig build run
```

---

12. Next steps

You've only touched the basics. To go deeper:

Bigger apps

- **`examples/mobus/`** — 26 endpoints, JWT auth, friends, WS chat, FCM
- **`examples/chat/`** — Durable Object SQLite + WebSocket

Individual topics

- **docs/handbook.md** — 15-minute overview (this tutorial compressed)
- **docs/db-backends.md** — SQLite / Turso / D1 internals + JSPI
- **docs/observability.md** — production metrics + logs
- **docs/benchmarks.md** / **benchmarks-long-run.md** — performance characteristics
- **docs/architecture.md** — framework internals + hardening notes

Model layer depth

- Relations (`has_many` / `belongs_to`)
- Eager loading (`am.model.preload.hasMany`) to kill N+1
- Custom validators
- Column renames (`__schema.columns`) — bridge `camelCase` Zig and `snake_case` SQL

- Multi-model + foreign keys

Community

- Issues / bugs → GitHub Issues
- Improvements → PRs welcome

Happy building.